

Laboratory of Decision Support Systems

Barbieri Bruno

Braghin Federica

Vannucci Nicholas

December 2025

Abstract

This is a concise abstract summarizing the motivation, method, and main results of the work.

1 Introduction

Introduce the problem, context, and contributions.

2 Related Work

Brief overview of relevant literature.

3 Methodology

Describe the proposed method or theoretical framework.

3.1 Assignment 4 – Data Warehouse Schema Design

L'Assignment 4 richiedeva il passaggio da un database operativo a un data warehouse orientato all'analisi, con l'obiettivo di progettare uno schema multidimensionale in grado di analizzare il numero di stream registrati a un mese dalla pubblicazione di una canzone (*Streams1Month*).

Lo schema fornito nella consegna era indicato come riferimento non vincolante, lasciando libertà progettuale nella definizione della granularità, delle dimensioni e delle relazioni. Seguendo l'approccio discusso a lezione, la progettazione è partita dall'analisi delle *business questions* e delle query che si intendevano supportare.

Da tale analisi è emerso che la misura centrale del data warehouse è unica:

- **Streams1Month**, ovvero il numero di stream registrati a un mese dalla pubblicazione del brano.

A partire da questa misura sono state individuate le dimensioni rilevanti per l'analisi:

- **DimSong**, che rappresenta la canzone pubblicata;
- **DimArtist**, che rappresenta gli artisti che hanno partecipato alla canzone;
- **DimDate**, che rappresenta la data di pubblicazione del brano;
- **DimArtistGeography**, che rappresenta il luogo di nascita dell'artista;
- **DimLyrics**, che rappresenta il testo del brano e le sue componenti;

- **DimSymphony**, che rappresenta le caratteristiche acustiche del brano;
- **DimAlbum**, che rappresenta il contesto editoriale della canzone, non necessariamente coincidente con un album tradizionale.

Un'ulteriore scelta progettuale rilevante è stata quella di non utilizzare le *natural key* presenti nei dataset originali, in quanto fortemente duplicate e poco affidabili. Sono state pertanto introdotte chiavi surrogate univoche, generate tramite UUID, al fine di garantire coerenza, stabilità e corretta identificazione delle dimensioni.

Evoluzione dello schema e scelte progettuali

In una prima fase progettuale è stata considerata una fact table denominata *PublishedSong*, con granularità definita come *una riga per canzone pubblicata da un artista*, collegata alle dimensioni Song, Artist, Date, Lyrics e Symphony.

In tale schema, la gestione degli artisti in featuring sarebbe stata possibile esclusivamente tramite l'introduzione di una *bridge table* tra *DimSong* e *DimArtist*. Tuttavia, questo approccio presentava alcune criticità rilevanti. In primo luogo, l'uso della bridge table avrebbe condotto a una struttura che non aderiva pienamente né allo *star schema* né allo *snowflake schema*, aumentando il numero di join necessari per l'esecuzione delle query analitiche e riducendo l'efficienza complessiva del sistema.

In secondo luogo, la gestione dei featuring risultava meno naturale dal punto di vista analitico, poiché il ruolo dell'artista (artista primario o featuring) non era direttamente associato alla misura analizzata, ma derivato da una struttura intermedia.

Sono state inoltre considerate altre alternative progettuali, tra cui:

- l'introduzione di un attributo *Role* nella bridge table;
- l'utilizzo di una *degenerate dimension* per distinguere tra artisti primari e featuring;
- la creazione di una dimensione di gruppo di artisti tra la fact table e la dimensione Artist.

Sebbene concettualmente corrette, tali soluzioni introducevano una maggiore complessità strutturale o rendevano meno immediate le analisi per artista, risultando meno adatte agli obiettivi dell'assignment.

Schema finale del Data Warehouse

La soluzione finale adottata è stata la creazione di una fact table denominata *FactParticipation*, che modella esplicitamente la partecipazione di un artista a una canzone.

La granularità finale è definita come:

una riga = partecipazione di un artista a una canzone in una determinata data.

Di conseguenza, la stessa canzone può comparire più volte all'interno della fact table, una volta per ciascun artista coinvolto. Questo comportamento è atteso ed è il risultato di una granularità più fine, scelta in modo consapevole per rappresentare correttamente la relazione molti-a-molti tra artisti e canzoni, evitando duplicazioni semantiche delle misure.

Il ruolo dell'artista è rappresentato tramite l'attributo *IsPrimary*, modellato come *degenerate dimension* all'interno della fact table:

- *IsPrimary* = 1 identifica l'artista principale;
- *IsPrimary* = 0 identifica gli artisti in featuring.

La fact table è collegata direttamente alle dimensioni *DimSong*, *DimArtist* e *DimDate*, mentre le dimensioni Song e Artist generano strutture a *snowflake schema* rispettivamente verso *DimAlbum*, *DimLyrics*, *DimSymphony* e *DimArtistGeography*. L'insieme delle foreign key identifica univocamente i record della fact table, in accordo con quanto discusso a lezione.

La dimensione *DimDate* è stata collegata direttamente alla fact table, in quanto rappresenta il riferimento temporale dell'evento analizzato (stream a un mese dalla pubblicazione). Altre date semanticamente differenti, come la data di rilascio dell'album, sono state mantenute come attributi all'interno delle rispettive dimensioni, evitando ambiguità e join aggiuntivi.

Implementazione

Una volta definito lo schema multidimensionale del Data Warehouse, è stata realizzata anche la sua implementazione fisica sul database server utilizzando SQL Server Management Studio. In particolare, è stato scritto ed eseguito uno script SQL per la creazione delle tabelle dimensionali e della fact table, in piena coerenza con lo schema logico descritto.

Le tabelle sono state inizialmente create vuote, definendo esplicitamente:

- gli attributi e i relativi tipi di dato;
- le chiavi primarie;
- le chiavi esterne che implementano le relazioni tra fact table e dimensioni.

Le foreign key riflettono correttamente la struttura del Data Warehouse, garantendo allineamento tra modello concettuale, logico e implementazione fisica. Sono stati inoltre introdotti vincoli e scelte sui tipi di dato per garantire coerenza e qualità dei dati. In particolare:

- l'attributo *Gender* della dimensione *DimArtist* è stato vincolato ai soli valori ammessi (*M* o *F*) tramite una *check constraint*;
- i tipi di dato numerici sono stati scelti in base alla natura degli attributi, utilizzando *INT* per conteggi discreti, *FLOAT* per misure continue e *BIT* per attributi booleani come *IsPrimary*;
- la misura *Streams1Month* è stata definita come attributo numerico adeguato a rappresentare valori di conteggio di grandi dimensioni.

Questa fase ha permesso di separare in modo chiaro la progettazione dello schema dalla fase di popolamento dei dati, che viene affrontata negli assignment successivi. Il caricamento effettivo dei dati nel Data Warehouse è infatti realizzato nell'Assignment 6, tramite strumenti ETL, a partire dai file CSV prodotti nell'Assignment 5.

3.2 Assignment 5 – Data Preparation and Dataset Splitting

L'Assignment 5 richiedeva lo sviluppo di un programma Python in grado di suddividere i dati di input in file distinti, uno per ciascuna dimensione e per la fact table del Data Warehouse progettato nell'Assignment 4, al fine di preparare i dataset per le successive fasi di caricamento.

Per soddisfare tale richiesta, il processo di *data preparation* è stato articolato in due fasi consecutive e complementari:

- pulizia e normalizzazione dei dati;
- suddivisione ed esportazione dei dataset, uno per ciascuna tabella del Data Warehouse.

Pulizia e normalizzazione dei dati

La prima fase è stata implementata tramite lo script `prepare_dw_datasets.py`, il cui obiettivo è rendere i dati coerenti con lo schema multidimensionale progettato.

Lo script carica i dataset in formato JSON dalla directory `dataset/correct_ids` ed esegue una serie di operazioni di pulizia e normalizzazione prima dello *split* vero e proprio. In particolare, vengono effettuate le seguenti operazioni:

- uniformazione dei tipi di dato (interi, float e stringhe);
- gestione consistente dei valori mancanti e non validi;
- normalizzazione degli attributi testuali;
- correzione di incongruenze sintattiche ereditate dai dataset originali;
- generazione e validazione delle chiavi surrogate, in linea con lo schema del Data Warehouse;
- normalizzazione della dimensione temporale e derivazione dell'attributo *Season*.

Il risultato di questa fase è un insieme di dataset puliti e coerenti, salvati nella directory `dataset/cleaned_json`, che rappresentano una versione intermedia pronta per essere suddivisa secondo le tabelle del Data Warehouse.

Esportazione

La seconda fase è stata implementata tramite lo script `export_dw_csv.py`, che realizza esplicitamente quanto richiesto dalla consegna, ovvero la suddivisione dei dati in file separati, uno per ciascuna tabella del Data Warehouse.

A partire dai dataset normalizzati, lo script genera un file CSV per ogni dimensione e per la fact table. Ogni file CSV contiene esclusivamente gli attributi previsti dallo schema del Data Warehouse ed è costruito in modo da rispettarne la granularità.

In particolare, la fact table *FactParticipation* mantiene la granularità definita nell'Assignment 4, ovvero:

una riga per partecipazione artista–canzone.

Essa include la misura *Streams1Month* e l'attributo *IsPrimary*, modellato come *degenerate dimension*.

I file CSV prodotti costituiscono quindi l'output finale dell'Assignment 5 e rappresentano l'input diretto per le successive fasi di caricamento del Data Warehouse.

L'Assignment 5 ha permesso di trasformare i dati di partenza in un insieme di file strutturati, uno per ciascuna tabella del Data Warehouse, come richiesto dalla consegna. La separazione tra fase di normalizzazione e fase di esportazione consente di ottenere dati puliti, coerenti e direttamente utilizzabili, facilitando le successive attività di popolamento e analisi.

3.3 Assignment 6 – Data Uploading with Python

L'Assignment 6 richiedeva lo sviluppo di un programma Python in grado di popolare il database del Data Warehouse (*Group_ID_8_DB*), rispettando lo schema e le relazioni definite nell'Assignment 4 e utilizzando i dati preparati negli assignment precedenti.

Poiché il caricamento completo dei dati può risultare oneroso in termini di tempo e risorse, è stato richiesto di progettare il codice in modo appropriato, tenendo conto sia delle prestazioni sia della stabilità del processo.

Il caricamento dei dati è stato implementato tramite uno script Python che utilizza la libreria `pyodbc` per stabilire una connessione a *SQL Server*. La connessione viene effettuata specificando

server, database e credenziali, consentendo l'esecuzione diretta di istruzioni SQL dal codice Python. Una volta stabilita la connessione, viene creato un cursore che permette di eseguire operazioni di inserimento sulle tabelle del Data Warehouse.

Per garantire il rispetto dei vincoli di integrità referenziale, le tabelle vengono caricate seguendo un ordine prestabilito, coerente con le relazioni di *foreign key* definite nello schema:

- dimensioni indipendenti;
- dimensioni che dipendono da altre dimensioni;
- fact table.

In particolare, le dimensioni vengono caricate prima della fact table *FactParticipation*, assicurando che tutte le chiavi esterne siano già presenti nel database al momento dell'inserimento dei fatti.

Prima di avviare il caricamento, lo script verifica se una tabella è già popolata tramite una semplice query di conteggio. Se una tabella risulta non vuota, il caricamento viene saltato, evitando duplicazioni accidentali dei dati. Questo controllo rende lo script riutilizzabile e consente di rilanciare il processo senza dover ripulire manualmente il database.

I dati vengono letti dai file CSV prodotti nell'Assignment 5. Per ogni tabella, lo script:

- legge l'intestazione del file per determinare le colonne;
- costruisce dinamicamente la query di inserimento;
- effettua una conversione esplicita dei tipi di dato per alcuni attributi critici, in particolare:
 - *Streams1Month*, convertito in valore numerico;
 - *IsPrimary*, convertito in valore booleano/integer;
- gestisce correttamente i valori mancanti, convertendoli in NULL.

Poiché il caricamento completo dei dati può richiedere tempi significativi, l'inserimento delle tuple è stato progettato in modalità *batch*. I record vengono accumulati in gruppi di dimensione fissa e inseriti nel database tramite operazioni di inserimento multiplo, seguite da un commit esplicito.

Questa scelta consente di ridurre il numero di comunicazioni tra Python e il database, migliorare le prestazioni complessive del caricamento e rendere il processo più stabile in presenza di grandi volumi di dati.

Lo script presentato rappresenta la versione finale del processo di caricamento: durante lo sviluppo sono stati effettuati diversi test e lanci intermedi per verificare la connessione al database e il corretto inserimento dei dati, prima di consolidare la soluzione a batch adottata.

L'Assignment 6 conclude quindi il processo di costruzione del Data Warehouse, collegando la fase di preparazione dei dati alla loro effettiva persistenza nel database in modo automatico, coerente e ripetibile.

3.4 Assignment 10 – Confronto regionale delle performance di streaming

L'Assignment 10 analizza il rapporto tra gli stream di una categoria musicale in una determinata regione e gli stream della stessa categoria nelle restanti regioni, considerando esclusivamente le canzoni in cui l'artista ricopre il ruolo di *primary artist*.

La logica analitica è stata inizialmente validata tramite una formulazione SQL al fine di verificarne la correttezza semantica e numerica, e successivamente implementata in *SQL Server Integration Services* (SSIS) mediante un unico *Data Flow*. Tale scelta garantisce coerenza con

lo schema del *Data Warehouse* e consente di mantenere un flusso di trasformazioni efficiente e facilmente tracciabile.

Il flusso dati ha origine dalla fact table *FactParticipation* ed è progressivamente arricchito tramite operazioni di *Lookup* sulle dimensioni *DimSong*, *DimArtist* e *DimArtistGeography*. Queste trasformazioni permettono di associare a ciascun record la categoria musicale del brano e la regione di nascita dell'artista principale. Un *Conditional Split* viene utilizzato per selezionare esclusivamente le righe relative ai *primary artist*, in accordo con la consegna.

L'aggregazione degli stream per ciascuna coppia (categoria, regione) è realizzata tramite una trasformazione di *Aggregate*. In parallelo, mediante una *Multicast*, viene calcolato il totale degli stream per categoria, valore necessario per determinare il volume di stream della stessa categoria nelle altre regioni.

I risultati intermedi vengono successivamente ricombinati attraverso una trasformazione di *Merge Join*, che consente di calcolare sia gli stream nelle restanti regioni sia il rapporto finale tra stream nella regione e stream nelle altre regioni. Tali calcoli sono implementati mediante trasformazioni di *Derived Column*, con una gestione esplicita dei casi di denominatore nullo.

Prima della scrittura dell'output finale, è stata introdotta una trasformazione di *Data Conversion* sulle colonne *Category* e *Region*. Questo passaggio è necessario poiché tali attributi provengono dalle dimensioni con tipo Unicode (*DT_WSTR*), mentre la *Flat File Destination* richiede stringhe non Unicode (*DT_STR*). La conversione garantisce la compatibilità dei tipi di dato e previene errori in fase di esecuzione, senza alterare il contenuto informativo.

3.5 Assignment 15 – Distribuzione mensile degli stream per regione e totale nazionale

L'Assignment 15 richiede di mostrare, per ciascun mese, il numero totale di stream registrati in ogni regione e il totale complessivo a livello nazionale per ciascun paese.

L'obiettivo dell'analisi è produrre, per ogni mese, una struttura ordinata che elenca tutte le regioni appartenenti a ciascun paese, seguite da una riga rappresentante il totale nazionale, riportando per ciascuna voce il numero di stream osservati. Per rendere esplicita e leggibile tale struttura nel risultato finale, sono stati introdotti due membri calcolati, denominati *Paese_Col* e *Regione_Col*, che consentono di visualizzare direttamente le etichette testuali del paese e della regione in ogni riga.

Questo approccio permette di rappresentare correttamente la riga di totale nazionale sostituendo la regione con una voce aggregata (*Total*), senza la necessità di creare nuovi membri all'interno della gerarchia geografica del cubo.

La costruzione dell'insieme completo delle righe è realizzata mediante la funzione *GENERATE*, che per ciascun paese produce dapprima l'elenco delle regioni associate, ottenute tramite la funzione *DESCENDANTS* limitata al livello *Region*, e successivamente una riga aggiuntiva corrispondente al totale nazionale. La combinazione di tale struttura con il livello *Month* della gerarchia temporale consente di replicare, per ogni mese, la sequenza *Paese-Regioni-Totale*.

La clausola *FILTER* è utilizzata per rimuovere eventuali membri vuoti o privi di nome, mentre l'uso di *NON EMPTY* consente di escludere automaticamente le combinazioni per cui il numero di stream risulta assente. Infine, l'ordinamento del risultato è ottenuto concatenando mese, paese e regione, così da garantire una presentazione coerente e facilmente interpretabile dei dati.

3.6 Assignment 16 – Impatto annuale degli stream per artista

L'Assignment 16 richiedeva di analizzare l'impatto annuale degli stream per artista, applicando una ponderazione differenziata in base al ruolo ricoperto (*Primary* o *Featured Artist*) e normalizzando il risultato sul totale degli stream dell'anno di riferimento. A tal fine, sono stati implementati i seguenti passaggi:

- **Ponderazione del ruolo (*WeightedStreams*)**. È stata definita una misura calcolata che applica un coefficiente pari a 0.8 agli stream associati agli artisti che ricoprono il ruolo di *Primary Artist* e un coefficiente pari a 0.2 nei casi di *Featured Artist*. Questa scelta consente di riflettere la diversa rilevanza degli artisti all'interno del contributo complessivo al successo dei brani.
- **Normalizzazione temporale (*GrandTotalYear*)**. Per calcolare la quota relativa, è stato definito un denominatore che isola il totale degli stream annuali del cubo, utilizzando il membro *All* sulle dimensioni *Artist* e *Role*. Questo approccio consente di svincolare il calcolo dai filtri di riga relativi al singolo artista e di confrontarne il contributo con l'intero mercato nell'anno di riferimento.
- **Calcolo della misura finale (*AvgYearlyArtistStreams*)**. La misura finale è calcolata come rapporto tra la somma degli stream pesati dell'artista e il totale degli stream dell'anno corrispondente. La definizione della misura include controlli tramite le funzioni *IIF* e *IsEmpty*, al fine di prevenire errori di divisione per zero e garantire la pulizia del risultato finale.

Dal punto di vista analitico, è stata applicata la clausola *NON EMPTY* per limitare l'analisi alle sole combinazioni Artista–Anno effettivamente attive, eliminando le occorrenze storiche prive di stream.

La percentuale ottenuta rappresenta una **quota annuale normalizzata** che esprime l'impatto reale di ciascun artista sul totale degli stream del mercato. Valori più elevati indicano una maggiore incidenza dell'artista come *Primary Artist*, mentre valori più contenuti sono tipici di artisti che partecipano prevalentemente come *Featured Artist*.

3.7 Assignment 17 – Variazione percentuale annuale degli stream per categoria

L'Assignment 17 richiedeva di analizzare la variazione percentuale, in aumento o in diminuzione, del numero di stream per ciascuna categoria musicale rispetto all'anno precedente.

La query MDX è stata progettata per confrontare il volume di stream di ogni categoria musicale (*[Song].[Category]*) nell'anno corrente con quello registrato nello stesso periodo dell'anno precedente. A tal fine, sono stati utilizzati i seguenti costrutti.

La funzione *PARALLELPERIOD* è stata impiegata per navigare la gerarchia temporale e recuperare i valori relativi all'anno precedente, mantenendo invariata la granularità dell'analisi. È stata inoltre definita una misura calcolata, denominata *Percentage Variation*, che automatizza il calcolo della variazione percentuale degli stream, includendo una gestione esplicita dei casi di divisione per zero tramite la funzione *IIF*.

Nella formulazione iniziale, la query produceva un numero elevato di valori *NULL* o infiniti, rendendo i risultati difficili da interpretare. Per migliorare la qualità dell'output, la soluzione è stata ottimizzata applicando una clausola *FILTER* combinata con *NOT IsEmpty*, così da rimuovere tutte le righe per cui non era possibile effettuare un confronto significativo. In particolare, sono stati esclusi i casi in cui non esiste un anno precedente di riferimento, come il primo anno presente nel database.

Questo filtraggio consente di isolare esclusivamente i record per i quali il confronto matematico è effettivamente definito. Di conseguenza, una variazione pari a -100% indica che una categoria musicale era presente nell'anno precedente ma non registra stream nell'anno corrente, rappresentando un'informazione di business reale. In assenza di tale filtraggio, non sarebbe stato possibile distinguere tra una categoria effettivamente scomparsa e una per cui mancano semplicemente dati storici.

Infine, l'ordinamento basato sulla *Member_Key* della dimensione temporale assicura una progressione cronologica coerente, facilitando la lettura e l'interpretazione dei trend storici.

Dall'output ottenuto emergono indicazioni rilevanti dal punto di vista del business. Variazioni percentuali fortemente positive segnalano l'emergere o l'esplosione di nuovi trend di mercato, mentre variazioni negative evidenziano fasi di contrazione che possono richiedere analisi strategiche dedicate. Un calo generalizzato nell'ultimo anno disponibile costituisce infine un indicatore tecnico della possibile incompletezza dei dati, dimostrando come la query sia sensibile alla completezza temporale del dataset.

4 Experiments

Explain experimental setup, data, and evaluation.

5 Results

Present and discuss results.

6 Conclusion

Summarize findings and outline future work.